Peblo AI Backend Engineer Challenge

Mini Content Ingestion + Adaptive Quiz Engine

Candidate Name: Vuskamalla Vyshnavi
Role: Backend AI Engineer
Date: March 2026

1. Introduction

In the modern digital learning ecosystem, educational platforms are rapidly evolving from static content delivery systems into intelligent and interactive learning environments. Traditionally, educational resources such as textbooks, worksheets, and lecture notes are distributed in formats like PDF documents. While these materials provide valuable information, they do not inherently support interactive learning experiences such as quizzes, adaptive assessments, or personalized learning paths.

Artificial Intelligence, particularly Large Language Models (LLMs), has introduced new possibilities for transforming static educational content into dynamic learning tools. By leveraging AI technologies, educational systems can automatically extract meaningful information from learning materials and convert them into engaging activities that help students test their knowledge and improve their understanding.

The goal of this project is to design and implement a backend system that demonstrates how AI can be used to automate the process of converting educational PDFs into structured quiz content. The system ingests PDF documents containing learning material, processes and structures the content, generates quiz questions using an AI model, and serves these quizzes through RESTful APIs.

2. Problem Statement

Educational platforms often rely on static learning materials such as PDFs, textbooks, and lecture notes. These materials provide knowledge but do not automatically support interactive assessments or personalized learning experiences.

Manually creating quizzes and assessments from educational content requires significant time and effort from educators. Additionally, traditional quizzes often lack adaptive capabilities, meaning that students receive the same questions regardless of their individual learning progress.

The challenge is to build a backend system that can automatically process educational content and convert it into interactive quizzes. The system should be able to ingest PDF documents, extract relevant information, generate quiz questions using AI, and provide APIs for quiz delivery and answer submission. Furthermore, the system should adjust the difficulty of questions based on student performance.

3. Objectives of the System

The main objectives of this project include:

• Designing a backend system capable of ingesting educational PDF documents
• Extracting and structuring content from PDF files
• Generating quiz questions using a Large Language Model (LLM)
• Storing structured content and questions in a database
• Providing RESTful APIs for quiz retrieval and answer submission
• Implementing adaptive quiz difficulty based on student responses

By achieving these objectives, the system demonstrates how AI can be integrated into educational platforms to automate the creation of interactive assessments.

4. System Architecture

The system architecture is designed as a modular backend pipeline that converts educational PDF documents into interactive quiz questions. The architecture follows a sequential processing workflow where each component performs a specific task in transforming raw learning materials into structured quiz data.

The system begins with a content ingestion module that accepts PDF documents containing educational material. Using a PDF processing library, the system extracts text from each page of the document. The extracted content is then cleaned and processed to remove unnecessary formatting or whitespace.

After text extraction, the system divides the content into smaller segments called content chunks. Chunking ensures that the AI model processes manageable portions of text while maintaining contextual relevance. Each chunk is assigned a unique identifier and stored along with metadata such as subject, topic, and source document reference.

The processed chunks are then passed to the AI quiz generation module, which integrates with a Large Language Model (LLM). The AI model analyzes the content and generates different types of quiz questions, including multiple-choice questions, true or false questions, and fill-in-the-blank questions. Each generated question maintains a reference to its original content chunk to ensure traceability.

All structured data, including content chunks, generated quiz questions, and student responses, are stored in a database layer. The database enables efficient retrieval of quizzes and tracking of student performance.

The backend exposes several REST API endpoints that allow interaction with the system. These APIs enable functions such as ingesting PDF content, generating quizzes, retrieving questions, and submitting answers.

Finally, the system includes a basic adaptive learning module that adjusts quiz difficulty based on student responses. If a student answers correctly, the system increases the difficulty level; if the answer is incorrect, the difficulty decreases. This adaptive mechanism helps create a more personalized learning experience.

Overall, the architecture demonstrates a simple but scalable design for building AI-powered educational platforms that transform static learning materials into interactive assessments.

The system architecture is designed as a modular backend pipeline that transforms educational PDF documents into structured quiz questions. The architecture focuses on content ingestion, AI-powered quiz generation, data storage, and adaptive quiz delivery through RESTful APIs.

The system begins with the content ingestion module, which accepts educational PDF documents as input. A PDF processing library is used to extract textual content from each page of the document. The extracted text is cleaned to remove unnecessary formatting and whitespace. This ensures that the content is suitable for further processing and analysis.

5. Technology Stack

The system was implemented using modern backend technologies that support rapid development, scalability, and efficient AI integration. The chosen tools provide a reliable environment for building APIs, processing documents, generating AI-based quiz content, and storing structured data.

Backend Framework

The backend of the system is developed using Python with FastAPI. FastAPI is a modern, high-performance web framework used for building APIs quickly and efficiently. It provides automatic request validation, asynchronous support, and built-in API documentation through Swagger UI. FastAPI also helps in building clean and modular backend architectures which makes the system easier to maintain and extend.

Programming Language

Python is used as the primary programming language for this project. Python is widely used in AI and backend development because of its simplicity, large ecosystem of libraries, and strong community support. It also integrates easily with machine learning and natural language processing libraries.

Database

The system uses SQLite as the database for storing structured data. SQLite is a lightweight and serverless relational database that is easy to set up and ideal for prototype systems. The database stores various entities including source documents, extracted content chunks, generated quiz questions, and student answers. This structured storage allows efficient retrieval and tracking of quiz data.

PDF Processing

The pdfplumber library is used to extract textual content from PDF documents. It enables the system to read educational materials from PDFs and convert them into

raw text that can be processed further. This step is essential for transforming static learning content into structured information that can be used for quiz generation.

AI Integration

The system integrates with the Google Gemini API, which acts as the Large Language Model (LLM) for generating quiz questions. The AI model analyzes extracted content chunks and generates different types of questions such as multiple choice, true or false, and fill-in-the-blank questions. The AI integration allows automated generation of educational assessments based on the source material.

API Testing and Documentation

During development, Swagger UI and Postman are used to test and validate API endpoints. Swagger UI is automatically generated by FastAPI and provides an interactive interface for testing APIs directly from the browser. Postman is used for sending API requests and verifying responses during backend testing.

Environment and Dependency Management

The project uses Python virtual environments and pip for dependency management. This ensures that all required libraries and packages are properly isolated and can be easily installed using a requirements file.

6. Content Ingestion Pipeline

The content ingestion pipeline is responsible for processing educational PDF documents and converting them into structured text segments that can be used for AI-based quiz generation. This pipeline acts as the first stage of the system, where raw learning materials are transformed into usable data.

The ingestion process begins when a PDF file containing educational content is uploaded to the system. These PDF files may contain lessons, explanations, or learning materials from different subjects and grade levels. The system reads the document and prepares it for further processing.

Steps in the Ingestion Process

1. Accept PDF File Input

The system accepts a PDF document through the ingestion API or through the backend file input mechanism. Each uploaded document is assigned a unique source identifier so that the system can track the origin of the extracted content.

2. Extract Text from Each Page

Using the pdfplumber library, the system reads each page of the PDF file and extracts the textual content. This step converts the document from a visual format into raw text that can be processed programmatically.

3. Clean the Extracted Text

The extracted text may contain extra spaces, formatting characters, or unwanted line breaks. The system performs basic cleaning operations such as removing unnecessary whitespace, normalizing text formatting, and ensuring that the text is readable and structured.

4. Divide Text into Smaller Chunks

After cleaning the text, the system divides the content into smaller segments known as **content chunks**. Chunking is important because large language models perform better when processing smaller and more focused sections of text. Each chunk

represents a small piece of learning material that can be used to generate quiz questions.

5. Store Chunks in the Database

Each generated chunk is stored in the database along with metadata such as the source document ID, subject, grade level, topic, and chunk identifier. This structured storage allows the system to easily retrieve the content later when generating quiz questions.

Chunking helps the AI model process smaller segments of content more effectively while preserving the meaning and context of the original educational material.

Example Content Chunk


```
{
 "source_id": "SRC_001",
 "chunk_id": "SRC_001_CH_01",
 "grade": 1,
 "subject": "Math",
 "topic": "Shapes",
 "text": "A triangle has three sides and three angles."
}
```


Each chunk represents a manageable unit of learning material that can be analyzed by the AI model. This structured representation ensures that quiz questions generated by the system remain directly connected to the original educational content.

7. AI-Based Quiz Generation

After extracting and structuring content from the PDF documents, the system uses an AI model to automatically generate quiz questions. This stage is responsible for transforming raw educational content into interactive assessment material that can be used to evaluate a student's understanding of the topic.

The AI model receives a content chunk as input. Each chunk contains a small section of educational text that focuses on a specific concept or topic. By analyzing this content, the AI model identifies key facts, definitions, and relationships within the text and generates questions based on that information.

The system sends the content chunk to the Large Language Model (LLM) through an API request. The prompt instructs the AI model to generate structured quiz questions that are relevant to the learning material. The model then produces questions in a structured JSON format so they can be easily stored in the database and served through APIs.

Supported Question Types

The system supports multiple types of quiz questions in order to create a more engaging learning experience:

• Multiple Choice Questions (MCQ) – Students select the correct answer from multiple options.
• True or False Questions – Students determine whether a statement based on the content is correct.
• Fill in the Blank Questions – Students provide the missing word or phrase in a sentence.

These question types allow the system to test different levels of understanding, including factual recall and concept comprehension.

Question Generation Process

The quiz generation process follows these steps:

1.Retrieve a stored content chunk from the database.

2.Send the chunk text to the AI model using a structured prompt.

3.The AI analyzes the educational content.

4.The AI generates quiz questions related to the content.

5.The generated questions are formatted into JSON structure.

Questions are stored in the database along with metadata.

Example Generated Question


```
{
 "question": "How many sides does a triangle have?",
 "type": "MCQ",
 "options": ["2","3","4","5"],
 "answer": "3",
 "difficulty": "easy",
 "source_chunk_id": "SRC_001_CH_01"
}
```


Each generated question maintains a reference to the original content chunk from which it was created. This traceability ensures that every quiz question can be linked back to the exact educational material used to generate it.

Maintaining this relationship is important because it allows educators or system administrators to verify the correctness of generated questions and ensure that the quiz content accurately reflects the source learning material.

8. API Design

The backend system exposes a set of RESTful APIs that allow users or external applications to interact with the quiz platform. These APIs are responsible for handling key operations such as content ingestion, quiz generation, question retrieval, and student answer submission. The APIs are designed using FastAPI, which provides a clear structure for handling requests and responses.

Each API endpoint performs a specific task in the system pipeline and communicates with the database to retrieve or store information. The APIs also return structured JSON responses, making them easy to integrate with frontend applications or other services.

POST /ingest

This endpoint is responsible for processing PDF files and extracting educational content. When a PDF document is uploaded, the system reads the file, extracts text from each page, cleans the text, and divides it into smaller content chunks. These chunks are then stored in the database for further processing.

This step acts as the starting point of the content ingestion pipeline.

POST /generate-quiz

This endpoint triggers the AI quiz generation process. The system retrieves stored content chunks from the database and sends them to the AI model for analysis. Based on the educational content in each chunk, the AI generates quiz questions such as multiple-choice questions, true or false statements, and fill-in-the-blank questions.

The generated questions are then saved in the database along with metadata such as difficulty level and source chunk reference.

GET /quiz

This endpoint allows users to retrieve quiz questions from the system. The API supports filtering questions based on parameters such as topic and difficulty level.

Example request:

GET /quiz?topic=shapes&difficulty=easy

The system searches the database and returns a list of quiz questions that match the specified criteria. This endpoint is useful for delivering quizzes dynamically based on the learner's current level.

Example response:


```
[
 {
  "question": "How many sides does a triangle have?",
  "options": ["2","3","4","5"],
  "difficulty": "easy"
 }
]
```


POST /submit-answer

This endpoint allows students to **submit their answers to quiz questions**. The request contains the student ID, question ID, and the selected answer.

Example request:


```
{
 "student_id": "S001",
 "question_id": "Q12",
 "selected_answer": "3"
}
```

9. Adaptive Difficulty Mechanism

The system includes a basic adaptive learning mechanism that dynamically adjusts the difficulty of quiz questions based on a student's performance. Adaptive learning helps personalize the learning experience by ensuring that students receive questions that match their knowledge level.

Difficulty Levels

The system categorizes quiz questions into three difficulty levels:

• **Easy** – Basic questions that test fundamental understanding of the topic.
• **Medium** – Moderately challenging questions that require deeper comprehension of the concept.
• **Hard** – Advanced questions that test critical thinking and detailed knowledge of the subject.

Adaptive Rules

The adaptive system follows simple rules to adjust the difficulty level of questions:

- **Correct Answer → Increase Difficulty**
  If a student answers a question correctly, the system increases the difficulty level for the next question.
- **Incorrect Answer → Decrease Difficulty**
  If the student answers incorrectly, the system decreases the difficulty level so that the learner can review easier concepts.

Adaptive Process Workflow

- The student retrieves a quiz question from the system.
- The student submits an answer using the API endpoint.
- The system evaluates the answer by comparing it with the correct answer stored in the database.
- Based on the result, the system adjusts the difficulty level for the next question.

10. Sample Output

The system returns quiz questions in structured JSON format through API responses. These responses contain the generated questions along with their options and other relevant metadata.

Example quiz response returned by the system:


```
[
 {
  "question": "How many sides does a triangle have?",
  "options": ["2","3","4","5"]
 },
 {
  "question": "A square has four sides.",
  "type": "True/False",
  "answer": "True"
 }
]
```


Each response contains quiz questions that are generated from the educational content extracted from the provided PDF documents.

The structured JSON format makes the response easy to integrate with frontend applications such as web platforms or mobile learning apps. It also allows the system to efficiently transmit quiz data between the backend server and client applications.

12. Conclusion

This project demonstrates the development of a simplified AI-powered backend system capable of transforming educational PDF documents into interactive quiz-based learning experiences.

The system integrates several important backend engineering concepts including content ingestion pipelines, AI-based question generation, structured database design, and RESTful API development. By combining these components, the system creates a workflow that converts static educational content into structured quizzes that can be accessed through APIs.

The implementation successfully processes raw PDF content, extracts meaningful text, generates quiz questions using an AI model, and stores the results in a structured database. The system also provides APIs that allow users to retrieve quizzes and submit answers, enabling interaction between students and the learning platform.

An additional feature of the system is the adaptive difficulty mechanism, which adjusts quiz difficulty based on student performance. This feature demonstrates how AI-based systems can personalize learning experiences and improve student engagement.

Although this implementation is designed as a prototype, it highlights the potential of AI-driven technologies in modern educational platforms. With further enhancements such as advanced adaptive learning algorithms, semantic search capabilities, and scalable cloud deployment, the system could evolve into a fully functional AI-powered learning platform.

Overall, this project illustrates how backend engineering and artificial intelligence can work together to convert traditional educational materials into interactive digital learning tools.